Missing Rate Limiting

1. Missing Rate Limiting on Contact Form API

OWASP Category

A04:2021 – Insecure Design

Affected Files

nginx configuration
/etc/nginx/sites-available/lt-nilavan

Application endpoint:
/api/sendgrid

Severity

High

Description

The /api/sendgrid endpoint s unlimited POST requests without any rate limiting or abuse prevention mechanism. The vulnerability exists because no request throttling controls were implemented at the application or reverse proxy level.

Business Impact

An attacker can flood the business owner's inbox with spam emails, exhaust the SendGrid email quota, cause operational disruption, and increase infrastructure costs.

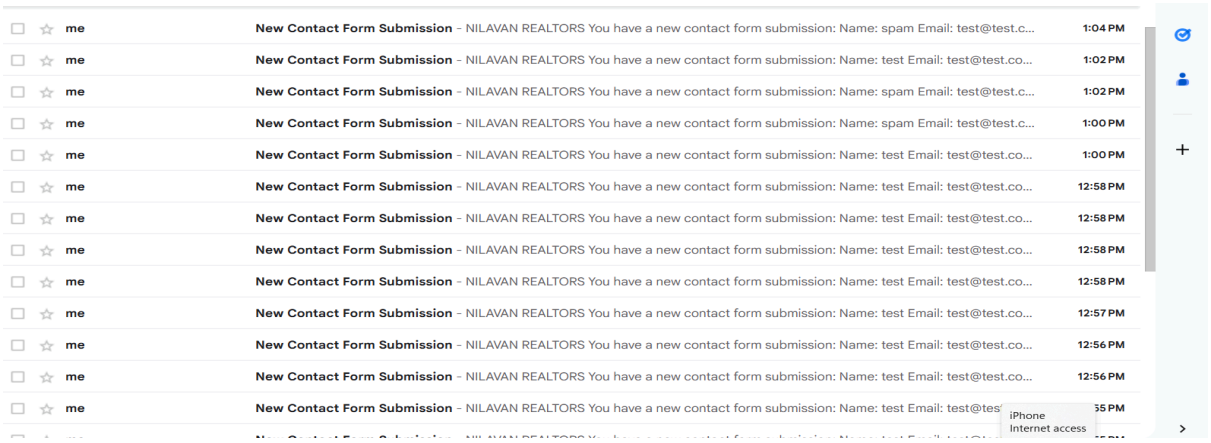
Proof of Concept

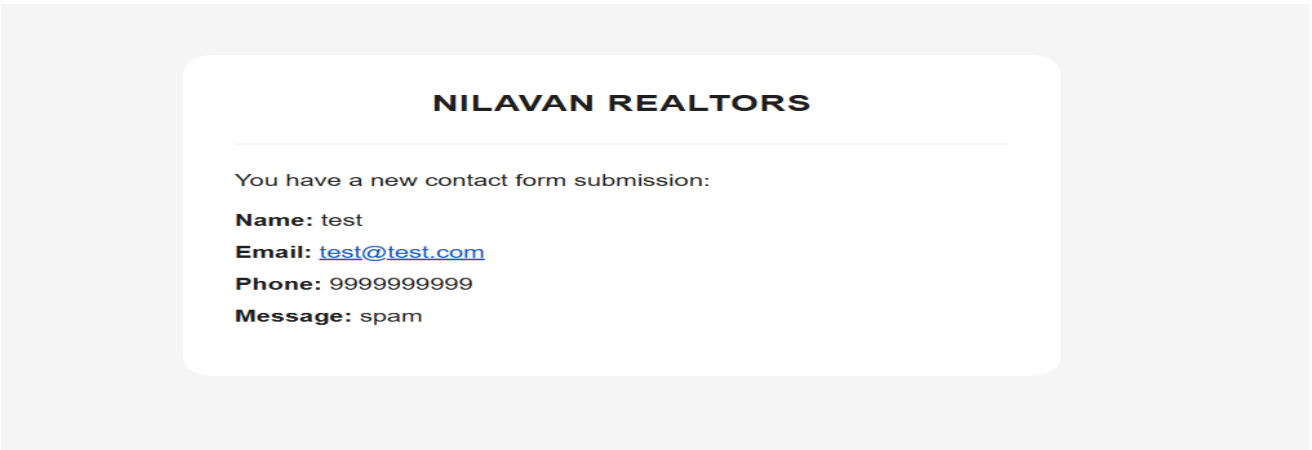
```
for i in $(seq 1 20); do
curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Content-Type: application/json" \
-d
'{"name":"spam","email":"test@test.com","phone":"9999999999","message":"
spam"}'
done
```

Result

All requests were accepted successfully without any restriction.

```
siva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$ for i in $(seq 1 20); do
curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Content-Type: application/json" \
-d '{"name":"spam","email":"test@test.com","phone":"9999999999","message":"spam message"}'
echo ""
done
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
{"success":true}
siva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$
```





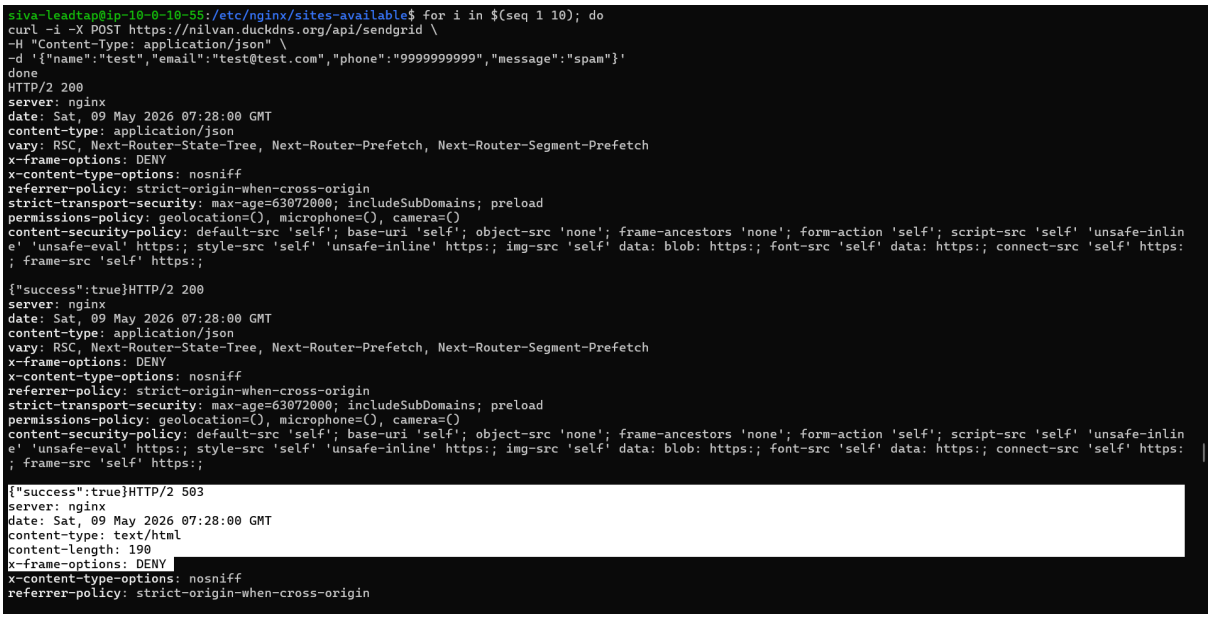
Recommended Fix

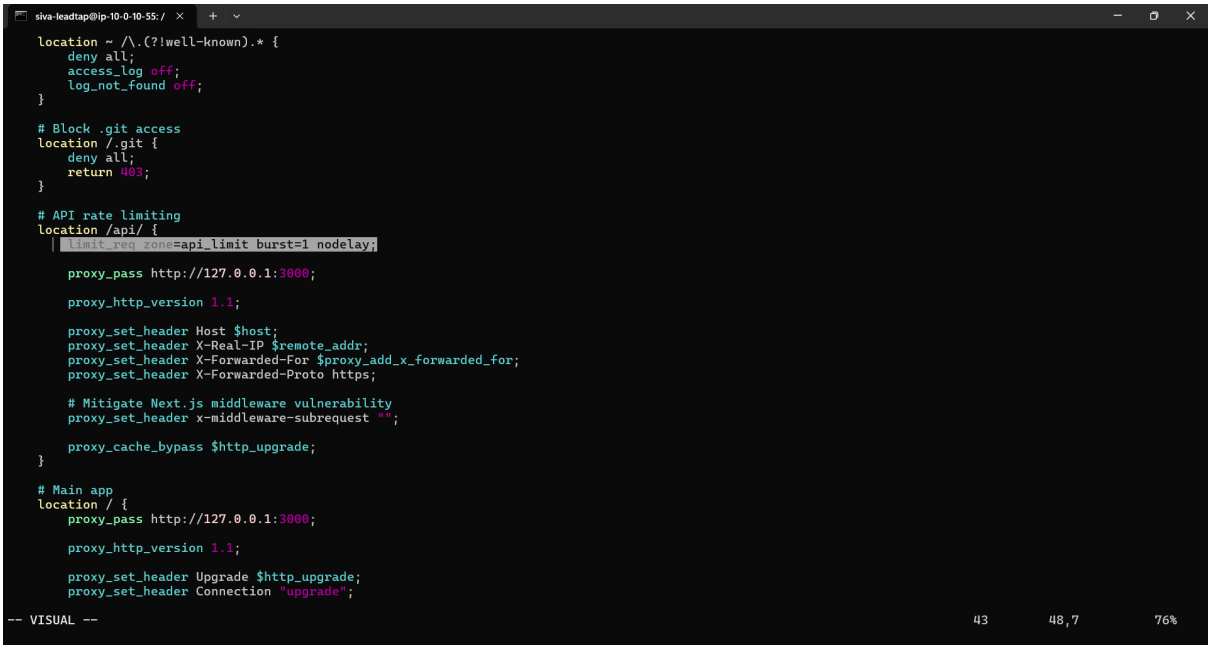
Implemented nginx rate limiting:

```
limit_req_zone $binary_remote_addr zone=sendgrid_limit:10m rate=2r/m;

location /api/sendgrid {
    limit_req zone=sendgrid_limit burst=1 nodelay;

    proxy_pass http://localhost:3000;
}
```





```
siva-leadtap@ip-10-0-10-55:/etc/nginx/sites-available$ cat cd ../nginx.conf
user www-data;
worker_processes auto;
worker_cpu_affinity auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
        worker_connections 768;
        # multi_accept on;
}

http {

        ##
        # Basic Settings
        ##

        sendfile on;
        tcp_nopush on;
        types_hash_max_size 2048;
        #server_tokens build; # Recommended practice is to turn this off

        # server_names_hash_bucket_size 64;
        # server_name_in_redirect off;
        server_tokens off;
        limit_req_zone $binary_remote_addr zone=api_limit:10m rate=2r/m;

        include /etc/nginx/mime.types;
        default_type application/octet-stream;

        ##
        # SSL Settings
        ##

        ssl_protocols TLSv1.2 TLSv1.3; # Dropping SSLv3 (POODLE), TLS 1.0, 1.1
```

HTML Injection

2. HTML Injection in Contact Form Email Template

OWASP Category

A03:2021 – Injection

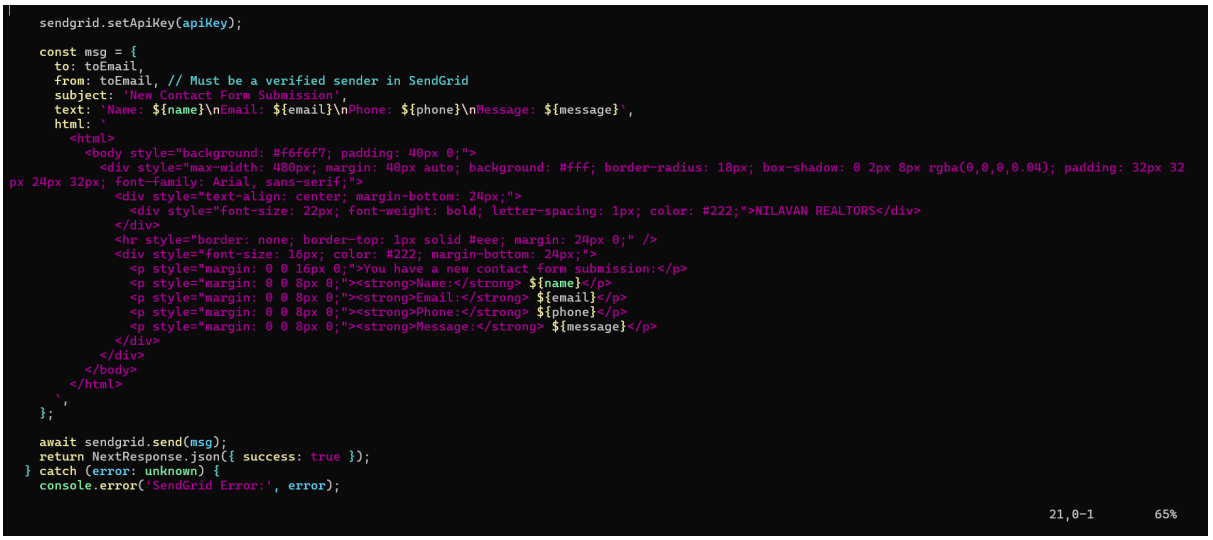
Affected File & Line Number

app/api/sendgrid/route.ts Lines 28-44

Severity: High

Affected code:

```
<p><strong>Name:</strong> ${name}</p>
<p><strong>Email:</strong> ${email}</p>
<p><strong>Phone:</strong> ${phone}</p>
<p><strong>Message:</strong> ${message}</p>
```



Description

The application directly inserts user-controlled input into the HTML email template without sanitization or escaping. Since the values are rendered inside the HTML body, malicious content such as clickable phishing links or manipulated formatting can be displayed to administrators receiving the email.

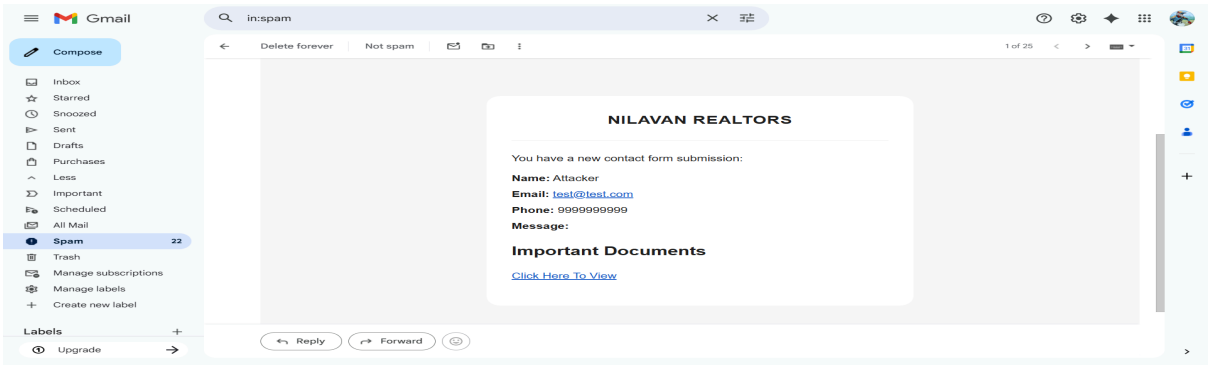
Business Impact

An attacker can:

- Inject malicious HTML into business emails
- Add phishing or scam links
- Manipulate email content viewed by staff
- Spoof trusted-looking messages
- Mislead employees into clicking attacker-controlled URLs

Proof of Concept

```
curl -X POST https://nilvan.duckdns.org/api/sendgrid \g/api/sendgrid \
-H "Content-Type: application/json" \
-d '{
"name":"Attacker",
"email":"test@test.com",
"phone":"9999999999",
"message":"<h2>Important Documents</h2><a
href=\"https://example.com\">Click Here To View</a>"
}'
{"success":true}
```



Resulting behaviour:

The received email rendered:

- **HACKED** in bold text
- A clickable external link inside the email body

This confirms that HTML content is executed instead of displayed as plain text.

Recommended Fix

Escape all user-controlled values before inserting them into HTML.

Example remediation:

```
const escapeHtml = (value: string) =>
  value
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
```

```
// 1. Helper function to prevent HTML Injection
const escapeHtml = (value: string) => {
  if (!value) return "";
  return String(value)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
};
```

Secure usage:

```
<p><strong>Name:</strong> ${escapeHtml(name)}</p>
<p><strong>Message:</strong> ${escapeHtml(message)}</p>
```

```
    // 6. Apply escapeHtml() to user variables in the HTML template (HTML Injection Fix)
    html: `
      <html>
        <body style="background: #f6f6f7; padding: 40px 0;">
          <div style="max-width: 480px; margin: 40px auto; background: #fff; border-radius: 18px; box-shadow: 0 2px 8p
x rgba(0,0,0,0.04); padding: 32px 32px 24px 32px; font-family: Arial, sans-serif;">
            <div style="text-align: center; margin-bottom: 24px;">
              <div style="font-size: 22px; font-weight: bold; letter-spacing: 1px; color: #222;">NILAVAN REALTORS</div
>
            </div>
            <hr style="border: none; border-top: 1px solid #eee; margin: 24px 0;" />
            <div style="font-size: 16px; color: #222; margin-bottom: 24px;">
              <p style="margin: 0 0 16px 0;">You have a new contact form submission:</p>
              <p style="margin: 0 0 8px 0;"><strong>Name:</strong> ${escapeHtml(name)}</p>
              <p style="margin: 0 0 8px 0;"><strong>Email:</strong> ${escapeHtml(email)}</p>
              <p style="margin: 0 0 8px 0;"><strong>Phone:</strong> ${escapeHtml(phone)}</p>
              <p style="margin: 0 0 8px 0;"><strong>Message:</strong> ${escapeHtml(message)}</p>
            </div>
          </div>
        </body>
      </html>
    `,
```

Missing Server-Side Input Validation

3. Missing Server-Side Input Validation

OWASP Category

A05:2021 – Security Misconfiguration

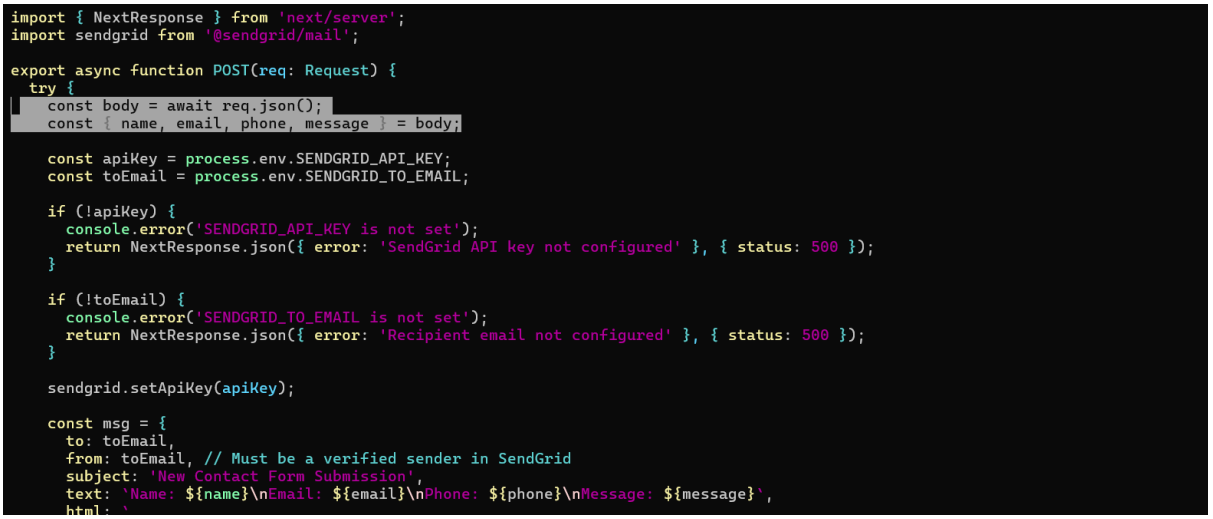
Affected File & Line Number

app/api/sendgrid/route.ts Lines 6-7

Severity: High

Affected code:

```
const body = await req.json();
const { name, email, phone, message } = body;
```



```
import { NextResponse } from 'next/server';
import sendgrid from '@sendgrid/mail';

export async function POST(req: Request) {
  try {
    const body = await req.json();
    const { name, email, phone, message } = body;

    const apiKey = process.env.SENDGRID_API_KEY;
    const toEmail = process.env.SENDGRID_TO_EMAIL;

    if (!apiKey) {
      console.error('SENDGRID_API_KEY is not set');
      return NextResponse.json({ error: 'SendGrid API key not configured' }, { status: 500 });
    }

    if (!toEmail) {
      console.error('SENDGRID_TO_EMAIL is not set');
      return NextResponse.json({ error: 'Recipient email not configured' }, { status: 500 });
    }

    sendgrid.setApiKey(apiKey);

    const msg = {
      to: toEmail,
      from: toEmail, // Must be a verified sender in SendGrid
      subject: 'New Contact Form Submission',
      text: `Name: ${name}\nEmail: ${email}\nPhone: ${phone}\nMessage: ${message}`,
      html: `
```

Description

The /api/sendgrid endpoint does not validate user input on the server side.

The API accepts any JSON data, including:

- Invalid email addresses
- Oversized payloads
- Non-string values
- Malicious content
- Empty fields

Validation currently exists only on the frontend form and can be bypassed easily using direct API requests.

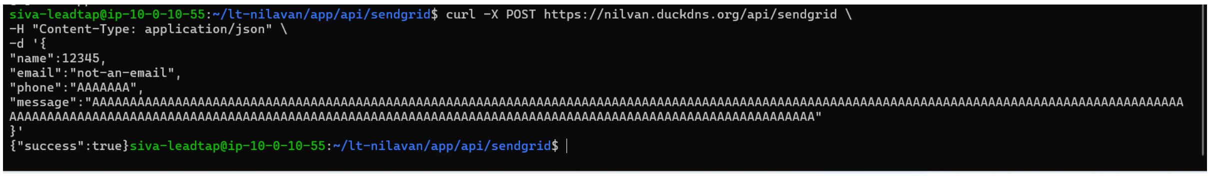
Business Impact

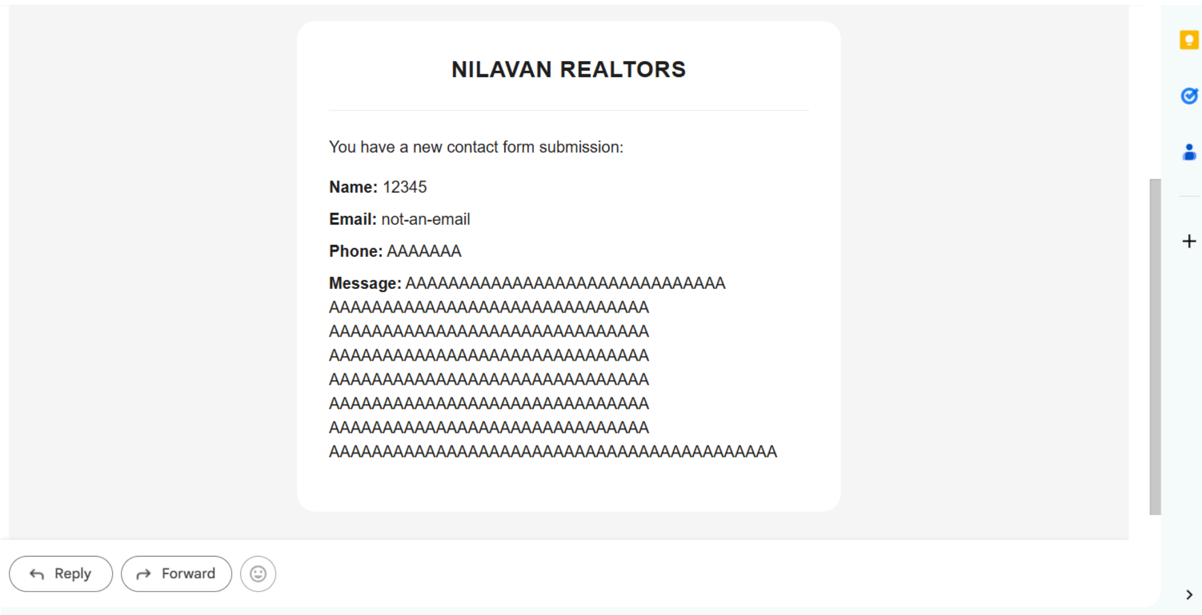
An attacker can:

- Send malformed requests directly to the API
- Cause email delivery failures
- Generate unnecessary server load
- Flood logs with invalid requests
- Abuse the endpoint using oversized payloads

Proof of Concept

```
curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Content-Type: application/json" \
-d '{
"name":12345,
"email":"not-an-email",
"phone":"AAAAAAA",
"message":"AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
}'
```





Resulting behaviour:

The API accepted invalid data and attempted to process the request instead of rejecting it with validation errors.

Recommended Fix

Implement strict server-side validation using Zod.

Example:

```
import { z } from "zod";

const ContactSchema = z.object({
  name: z.string().min(1).max(80),
  email: z.string().email().max(254),
  phone: z.string().min(7).max(20),
  message: z.string().min(1).max(1000),
});

const parsed = ContactSchema.safeParse(body);

if (!parsed.success) {
  return NextResponse.json(
    { error: "Invalid request" },
    { status: 400 }
  );
}
```

```
// 2. Define strict Zod validation schema
const ContactSchema = z.object({
  name: z.string().min(1, "Name is required").max(80),
  email: z.string().email("Invalid email format").max(254),
  phone: z.string().min(7, "Phone number is too short").max(20),
  message: z.string().min(1, "Message is required").max(2000),
});
```

Broken Access Control

No Origin / Referer Validation on Contact Form API

OWASP Category

A01:2021 – Broken Access Control

Affected File & Line Number

app/api/sendgrid/route.ts Line 5

Severity

Medium

Description

The /api/sendgrid endpoint accepted requests from any origin without validating trusted frontend domains.

The API did not verify:

- Origin header
- Referer header
- trusted frontend sources

As a result, attackers could directly interact with the API using curl, Postman, scripts, or malicious external websites.

The vulnerability existed because the backend trusted all incoming requests without validating whether they originated from the official application domain.

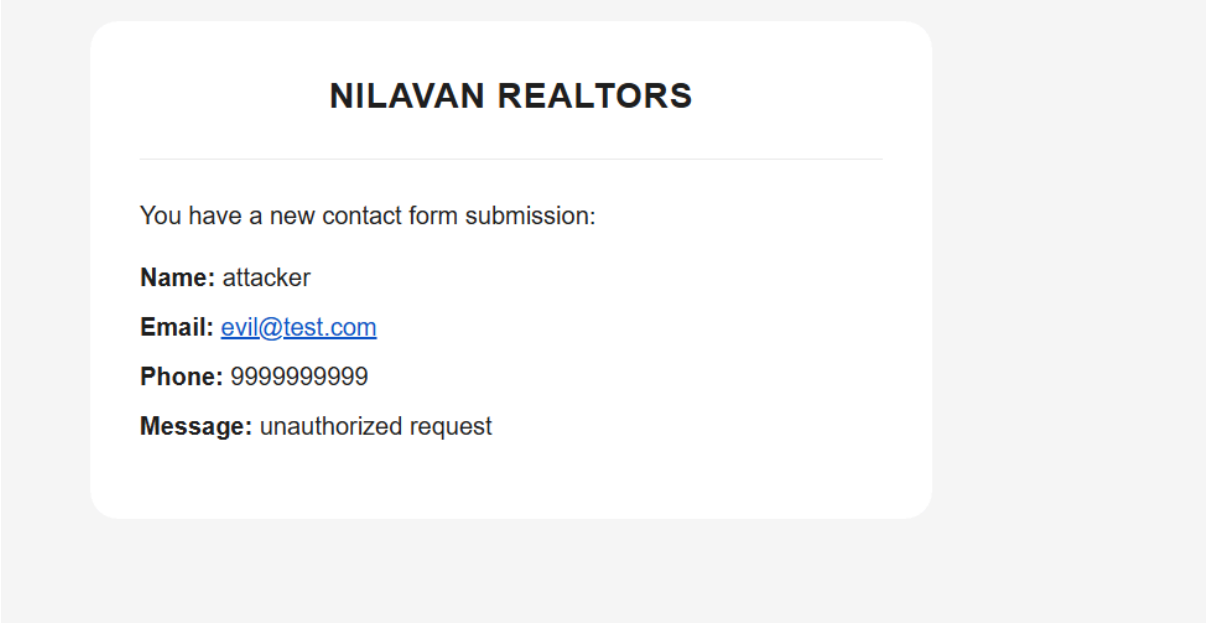
Business Impact

An attacker can:

- Abuse the public API from external websites
- Automate spam submissions
- Bypass frontend restrictions
- Trigger unauthorized backend requests

Proof of Concept

```
curl -X POST https://nilvan.duckdns.org/api/sendgrid \
-H "Origin: https://evil.com" \
-H "Referer: https://evil.com" \
-H "Content-Type: application/json" \
-d '{
"name":"attacker",
"email":"evil@test.com",
"phone":"9999999999",
"message":"unauthorized request"
}'
```



Result

The request was accepted successfully even though it originated from an untrusted domain.

Recommended Fix

Implemented trusted origin validation inside:

```
app/api/sendgrid/route.ts
```

Remediation:

```
const origin = req.headers.get("origin");
const referer = req.headers.get("referer");

const allowedOrigin = "https://nilvan.duckdns.org";

if (
  origin !== allowedOrigin ||
  !referer?.startsWith(allowedOrigin)
) {
  return NextResponse.json(
    { error: "Forbidden" },
    { status: 403 }
  );
}
```

Verification

After remediation:

Requests from untrusted domains returned:

HTTP 403 Forbidden

Only requests originating from the official frontend domain were accepted successfully.

Vulnerable and Outdated Dependencies

5. Vulnerable and Outdated Dependencies

OWASP Category

A06:2021 – Vulnerable and Outdated Components

Severity

Critical / High

Description

The application contains multiple outdated third-party dependencies identified through npm audit. Several installed packages have publicly disclosed security vulnerabilities affecting the application security posture.

Affected Components

- next
- axios
- lodash
- form-data
- glob
- postcss

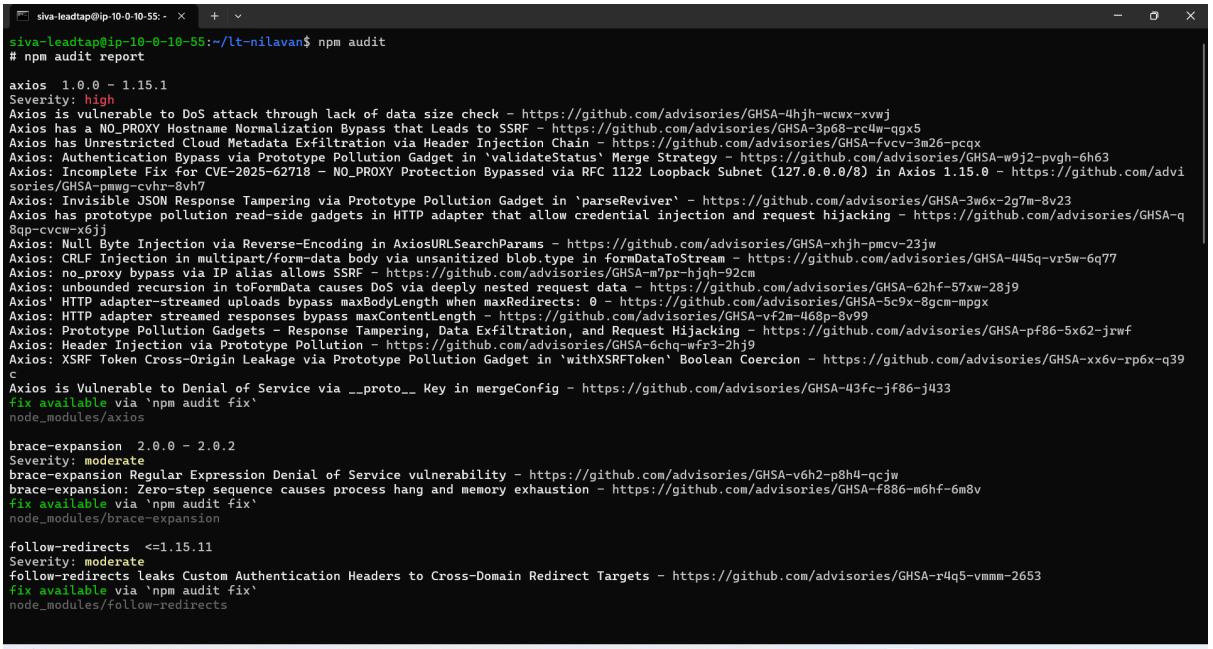
Business Impact

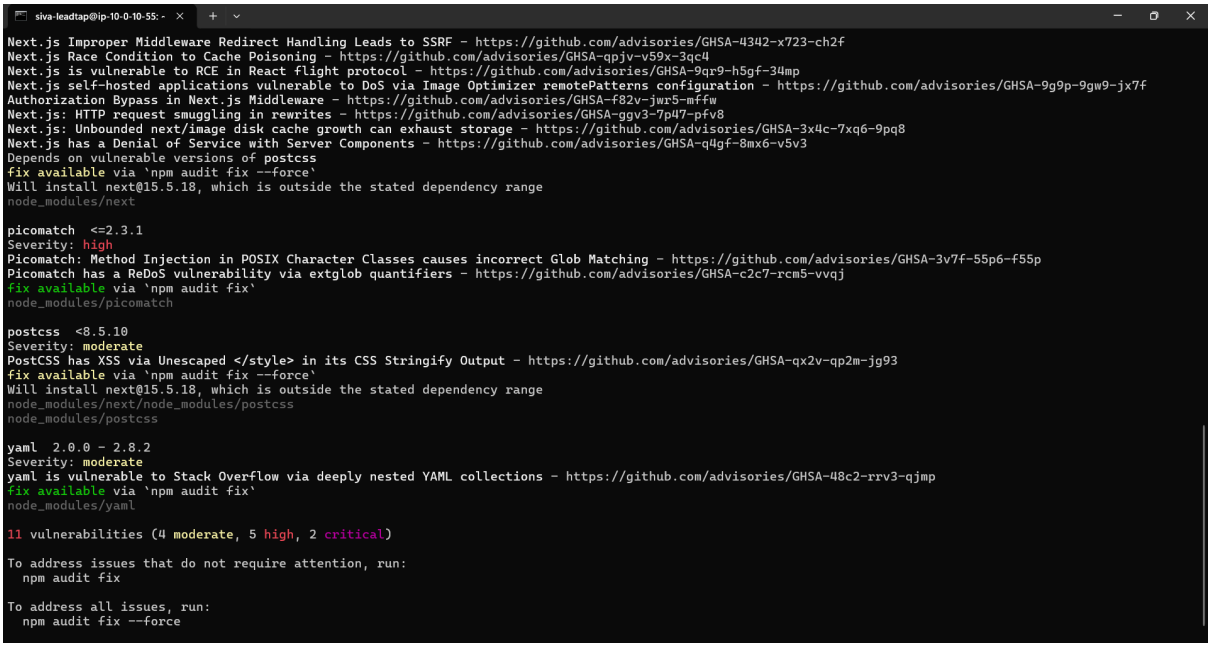
Exploitation of vulnerable dependencies may lead to:

Several attacks

Proof of Concept

npm audit





Result

11 vulnerabilities detected
2 Critical
5 High
4 Moderate

Recommended Fix

npm audit fix
npm install next@latest
npm update

Verification

npm audit

Expected Result:

found 0 vulnerabilities